

pdf version

Absolutely, Radoslaw — I can prepare a **course-ready, PDF-style version** of the 30 prompts (educational + ultra-safe + aggressive for all 10 refactoring groups), written in clean, structured, professional English.

Since I cannot directly generate a PDF file, I'll produce a **perfectly formatted, PDF-ready document** that you can paste into any PDF generator (Notion, Google Docs, Word, Canva, etc.) and export instantly.

Below is the **full course version**, formatted as a standalone document.

AI-Refactorable 100% — Module-Level Refactoring Prompts

Educational, Ultra-Safe, and Aggressive Variants for All 10 Refactoring Categories

Course Edition — English Version

Introduction

This document contains **30 specialized prompts** designed for module-level or package-level refactoring using AI.

Each refactoring category includes:

1. **Educational Prompt** — AI explains, teaches, and reviews without modifying code.
2. **Ultra-Safe Prompt** — AI performs only 100% deterministic, risk-free refactoring.
3. **Aggressive Prompt** — AI performs broad, deep, structural refactoring while preserving logic.

These prompts are ideal for:

- developer education,
- code quality workshops,
- automated refactoring pipelines,
- modernization projects,
- AI-assisted code reviews.

Naming (Technical, Non-Domain)

Educational Prompt

Analyze naming conventions in this module/package. Do NOT modify the code.

Explain:

- ◇ unclear or abbreviated names,
- ◇ generic method names,
- ◇ boolean naming issues,
- ◇ variable names that do not reflect usage.

Provide improved alternatives and before/after examples.

Ultra-Safe Prompt

Refactor naming using only 100% safe, technical transformations:

- ◇ expand abbreviations (usr → user),
- ◇ improve boolean names (active → isActive),
- ◇ replace generic method names (check → isValid),
- ◇ improve variable names based on type and usage.

Do NOT rename domain concepts or public APIs unless guaranteed safe.

Return refactored code + list of changes.

Aggressive Prompt

Perform an aggressive naming refactor across this module/package:

- ◇ expand all abbreviations,
- ◇ rename all generic methods to precise technical names,
- ◇ unify naming conventions across files,
- ◇ rename helper/manager/utils classes to meaningful technical names,
- ◇ apply consistent boolean naming.

Do NOT change business logic.

Return full refactored code + rename map.

Logic Simplification

Educational Prompt

Analyze logic complexity. Do NOT modify code.

Explain:

- ◇ where guard clauses should be used,
- ◇ where boolean extraction improves readability,
- ◇ where negations should be eliminated,
- ◇ where boolean flags are harmful.

Provide examples.

Ultra-Safe Prompt

Simplify logic using only safe transformations:

- ◇ convert nested if-statements to guard clauses,
- ◇ extract complex boolean expressions,
- ◇ eliminate negations when unambiguous,
- ◇ simplify boolean comparisons.

Do NOT change logic.

Return refactored code + list of changes.

Aggressive Prompt

Perform aggressive logic simplification:

- ◇ rewrite all nested conditionals into guard clauses,
- ◇ extract all multi-condition expressions,
- ◇ remove boolean flags (split methods),
- ◇ unify logical structure across files.

Do NOT change business logic.

Return full refactored code.

Code Structure

Educational Prompt

Review structural quality. Do NOT modify code.

Explain:

- ◇ magic numbers,
- ◇ dead code,
- ◇ println usage,
- ◇ inconsistent class structure,
- ◇ anonymous classes that should be lambdas.

Ultra-Safe Prompt

Apply safe structural refactoring:

- ◇ extract magic numbers used multiple times,
- ◇ remove unused imports and commented code,
- ◇ replace println with logger,
- ◇ reorder class elements safely,
- ◇ convert trivial anonymous classes to lambdas.

Return refactored code.

Aggressive Prompt

Perform aggressive structural cleanup:

- ◇ extract all magic numbers,
- ◇ remove all dead code,
- ◇ enforce consistent class structure,
- ◇ convert all anonymous classes to lambdas,
- ◇ convert lambdas to method references,
- ◇ unify logger usage.

Return full refactored code.

Null Safety

Educational Prompt

Analyze null-safety issues. Do NOT modify code.

Explain:

- ◇ missing null checks,
- ◇ missing @NotNull/@Nullable,
- ◇ dangerous null returns,
- ◇ where Optional is appropriate.

Ultra-Safe Prompt

Apply safe null-safety improvements:

- ◇ add @NotNull/@Nullable based on usage,
 - ◇ replace return null with empty collections,
 - ◇ add Objects.requireNonNull() at boundaries.
- Do NOT introduce Optional unless already used.
Return refactored code.

Aggressive Prompt

Perform aggressive null-safety refactoring:

- ◇ annotate all parameters and returns,
 - ◇ replace all null returns with safe alternatives,
 - ◇ convert null-handling to Optional where appropriate,
 - ◇ enforce fail-fast validation everywhere.
- Return full refactored code.

Immutability

Educational Prompt

Explain immutability issues. Do NOT modify code.

Identify:

- ◇ missing final,
- ◇ mutable collections,
- ◇ unnecessary wrappers,
- ◇ mutable fields.

Ultra-Safe Prompt

Apply safe immutability improvements:

- ◇ add final where variables are never reassigned,
- ◇ replace new ArrayList<>() with List.of() when list is never modified,

- ◇ replace wrappers with primitives when null is impossible.
- Return refactored code.

Aggressive Prompt

Perform aggressive immutability refactoring:

- ◇ enforce final everywhere,
- ◇ convert all immutable collections to List.of()/Map.of(),
- ◇ convert wrappers to primitives,
- ◇ mark fields final when safe.

Return full refactored code.

Formatting & Style

Educational Prompt

Review formatting and style. Do NOT modify code.

Explain:

- ◇ inconsistent indentation,
- ◇ missing blank lines,
- ◇ inconsistent logger usage,
- ◇ import issues.

Ultra-Safe Prompt

Apply safe formatting improvements:

- ◇ unify indentation,
- ◇ add blank lines between logical blocks,
- ◇ clean up imports,
- ◇ unify bracket style.

Return refactored code.

Aggressive Prompt

Apply aggressive style unification:

- ◇ enforce consistent formatting across all files,
- ◇ unify logger usage,
- ◇ unify naming of constants,
- ◇ enforce consistent method ordering.

Return full refactored code.

Exceptions

Educational Prompt

Explain exception-handling issues. Do NOT modify code.

Identify:

- ◇ empty catch blocks,
- ◇ overly broad catches,
- ◇ unnecessary try/catch,
- ◇ missing logging.

Ultra-Safe Prompt

Apply safe exception improvements:

- ◇ replace empty catch with logging,
- ◇ narrow catch types when obvious,
- ◇ remove unnecessary casts.

Return refactored code.

Aggressive Prompt

Perform aggressive exception refactoring:

- ◇ rewrite all empty catches,
- ◇ narrow all broad catches,
- ◇ remove redundant try/catch,
- ◇ unify exception-handling style.

Return full refactored code.

Annotations & Boilerplate

Educational Prompt

Explain annotation and boilerplate issues. Do NOT modify code.

Identify:

- ◇ missing @Override,
- ◇ manual getters/setters,
- ◇ manual equals/hashCode,
- ◇ missing constructors.

Ultra-Safe Prompt

Apply safe annotation improvements:

- ◇ add @Override,
- ◇ replace trivial getters/setters with Lombok,
- ◇ replace trivial equals/hashCode with Lombok.

Return refactored code.

Aggressive Prompt

Perform aggressive boilerplate cleanup:

- ◇ convert all getters/setters to Lombok,
- ◇ convert all builders to @Builder,
- ◇ convert all equals/hashCode to Lombok,
- ◇ generate constructors.

Return full refactored code.

Loops & Collections

Educational Prompt

Explain loop and collection issues. Do NOT modify code.

Identify:

- ◇ index loops,
- ◇ raw types,
- ◇ mutable collections,
- ◇ missing generics.

Ultra-Safe Prompt

Apply safe loop/collection improvements:

- ◇ replace index loops with for-each when index unused,
- ◇ add generics,
- ◇ add diamond operator.

Return refactored code.

Aggressive Prompt

Perform aggressive loop/collection refactoring:

- ◇ convert loops to streams,
- ◇ replace mutable collections with immutable ones,
- ◇ enforce generics everywhere.

Return full refactored code.

Comments & Documentation

Educational Prompt

Explain documentation issues. Do NOT modify code.

Identify:

- ◇ redundant comments,
- ◇ commented-out code,
- ◇ unclear TODOs,
- ◇ missing Javadoc.

Ultra-Safe Prompt

Apply safe documentation cleanup:

- ◇ remove commented-out code,
- ◇ remove comments describing “what”,
- ◇ normalize TODO/FIXME.

Return refactored code.

Aggressive Prompt

Perform aggressive documentation refactoring:

- ◇ remove all redundant comments,
- ◇ rewrite unclear TODOs,
- ◇ improve Javadoc,
- ◇ enforce consistent documentation style.

Return full refactored code.

End of Document

If you'd like, I can now prepare:

- ◇ a **cover page**,
- ◇ a **table of contents**,
- ◇ a **visual layout** (for Notion/Canva),
- ◇ or a **condensed cheat-sheet version**.

Just tell me what you want next.